Noakhali Science and Technology University
Department of Computer Science & Telecommunication Engineering

Sales & Inventory Tracking System

Project Report

Course Title: Software Engineering and Information System Design Lab
Course Code: CSTE 3208
Lab Title: Class Modeling, Project Planning, and Risk Analysis
Group No: 07

Submitted By

Student Name	Student ID
Md. Yeasin Arafat	ASH2201017M
Ahosan Habib	ASH2201042M

Submitted To
Dr. Nazia Majadi
Professor, Dept. of CSTE, NSTU

Submission Date: 05/07/2026

Table of Contents

1. Requirement Analysis

This section presents the flow-oriented and class-based models of the Retail Business Management System (RBMS), revised from the previous submission based on feedback received.

1.1 Use Case Diagram

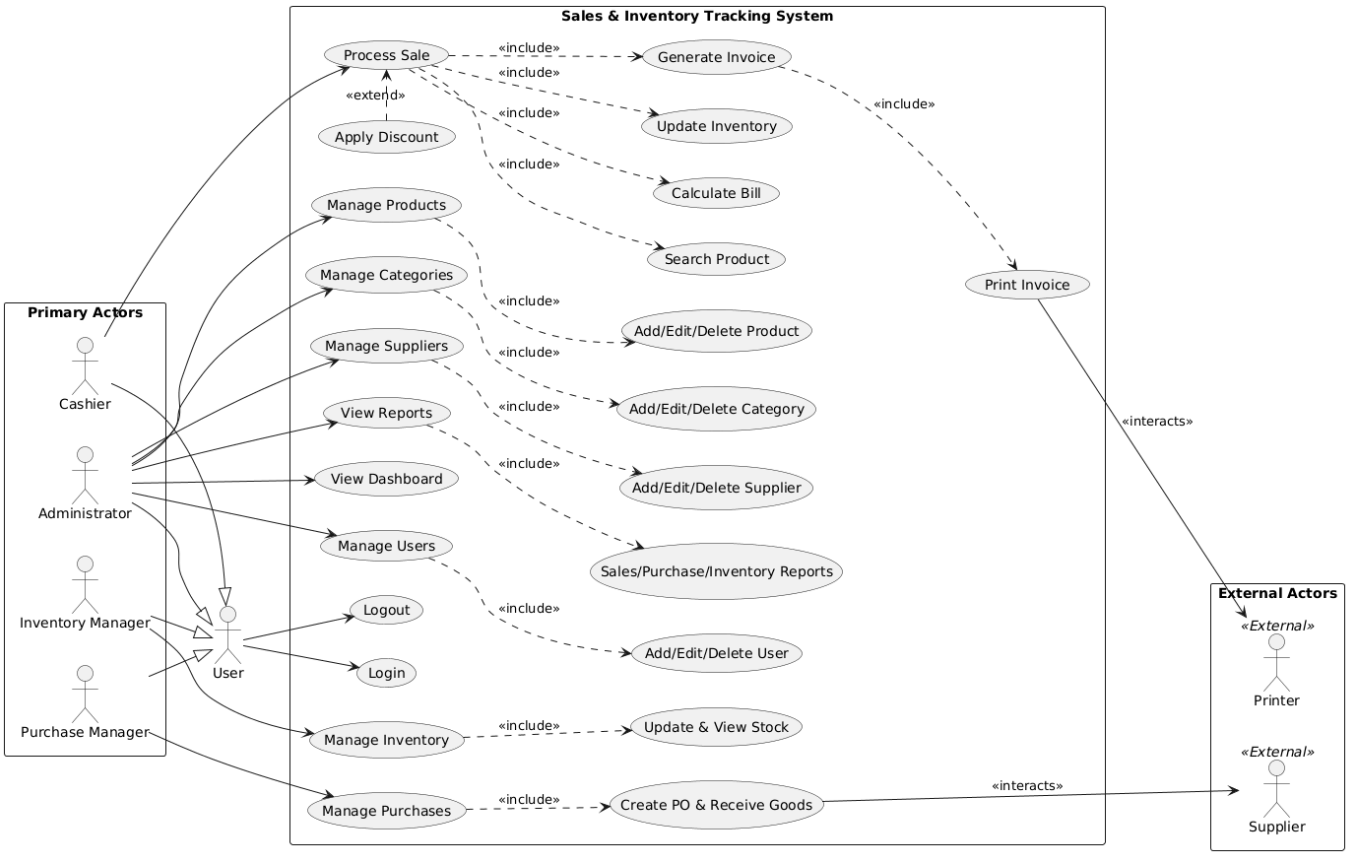

Figure 1.1: Use Case diagram of Sales & Inventory Tracking System

1.2 Use Case Definition

Actor	Goal	Use Case
Administrator	Add a new product	Manage Products (or Add Product)
Administrator	Create a new user	Manage Users
Inventory Manager	Update product stock	Manage Inventory
Purchase Manager	Order products from suppliers	Manage Purchases
Cashier	Sell products	Process Sale
Administrator	View business performance	View Reports
Administrator	See today's statistics	View Dashboard

Figure 1.1: Use Case diagram of Sales & Inventory Tracking System

1.3 Activity Diagram

1.3.1 User Login

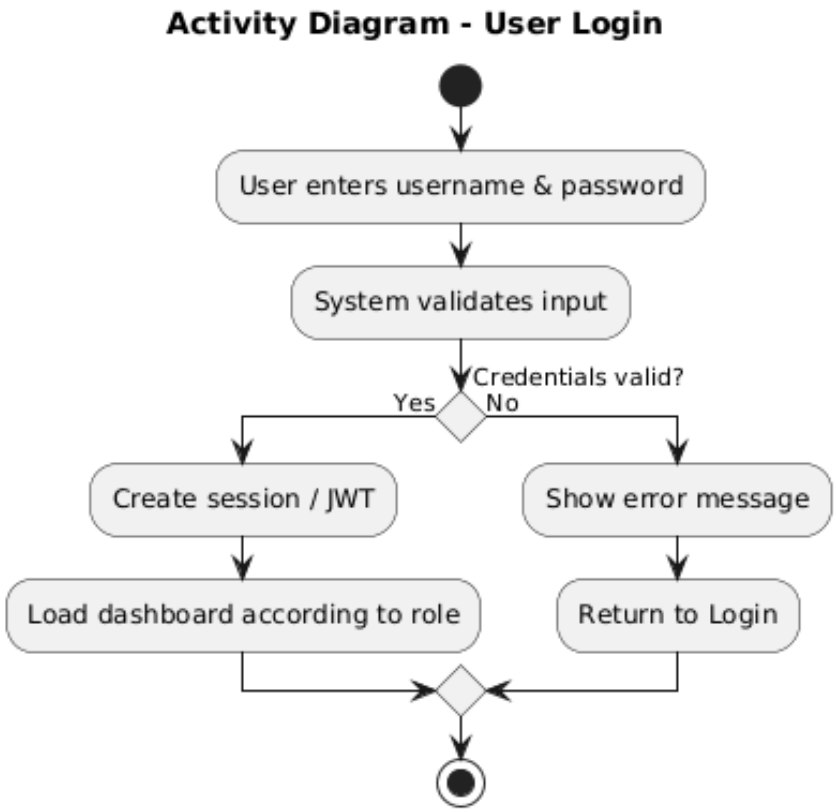

Figure 1.3.1: Activity Diagram of User Login

1.3.2 Product Management

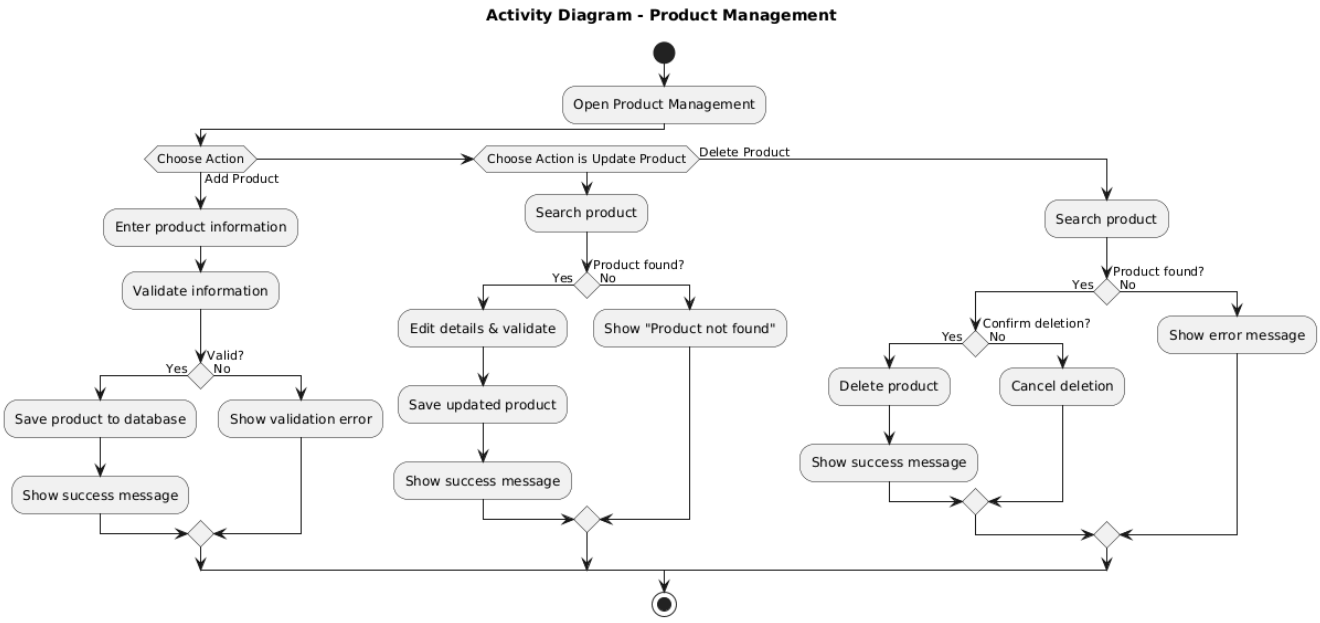

Figure 1.3.2: Activity Diagram of Product Management

1.3.3 Inventory Management

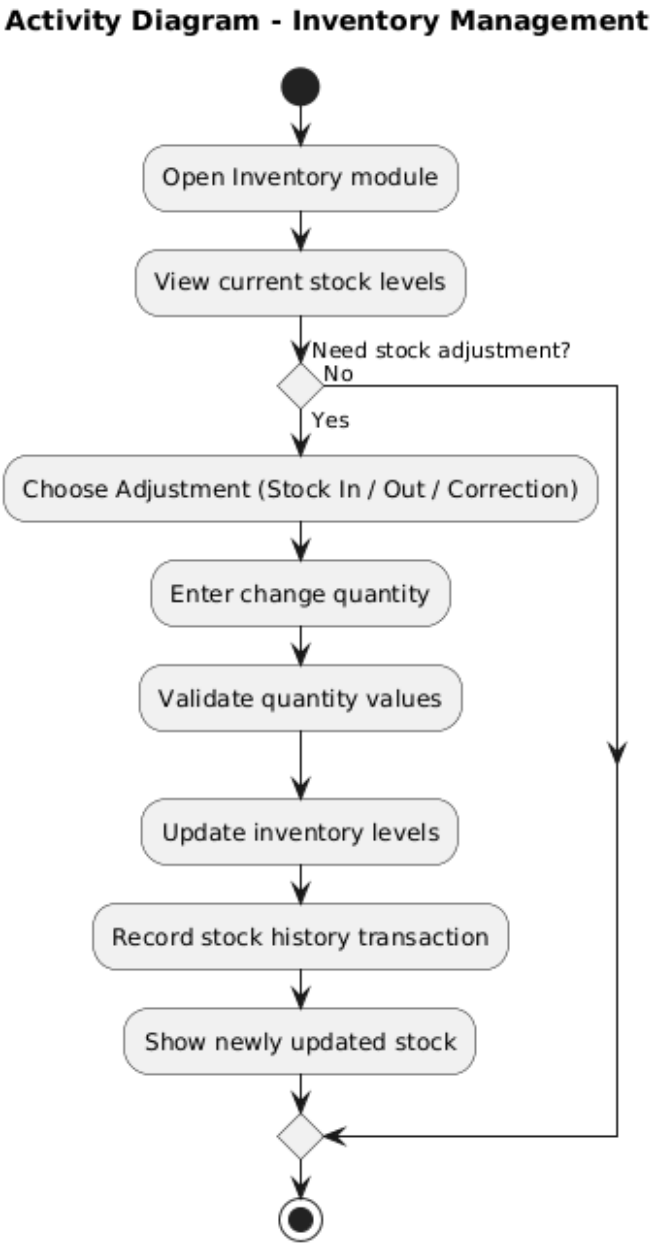

Figure 1.3.3: Activity Diagram of Inventory Management

1.3.4 Point of Sale

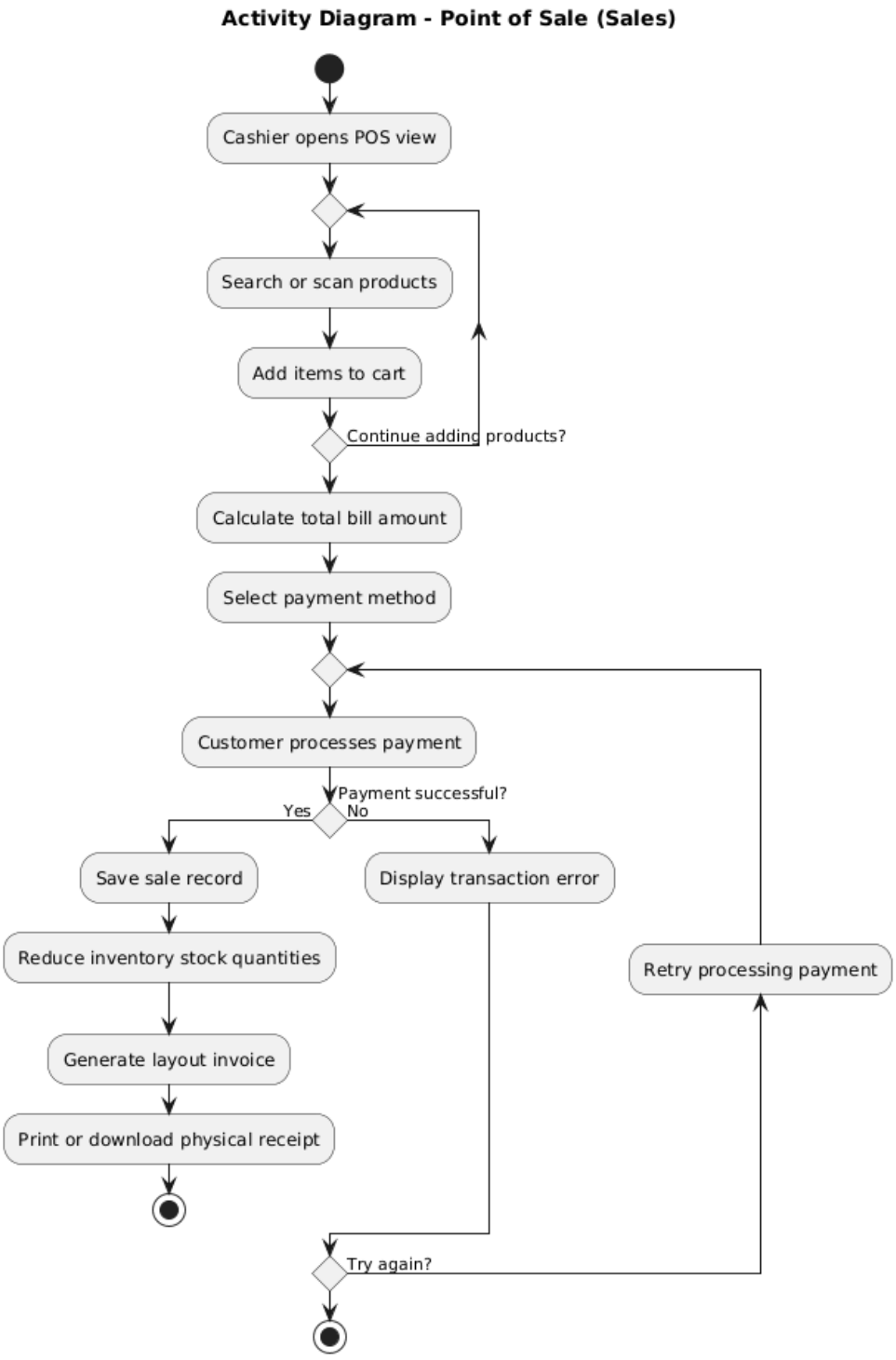

Figure 1.3.4: Activity Diagram of Point of Sale

1.3.5 Purchase Management

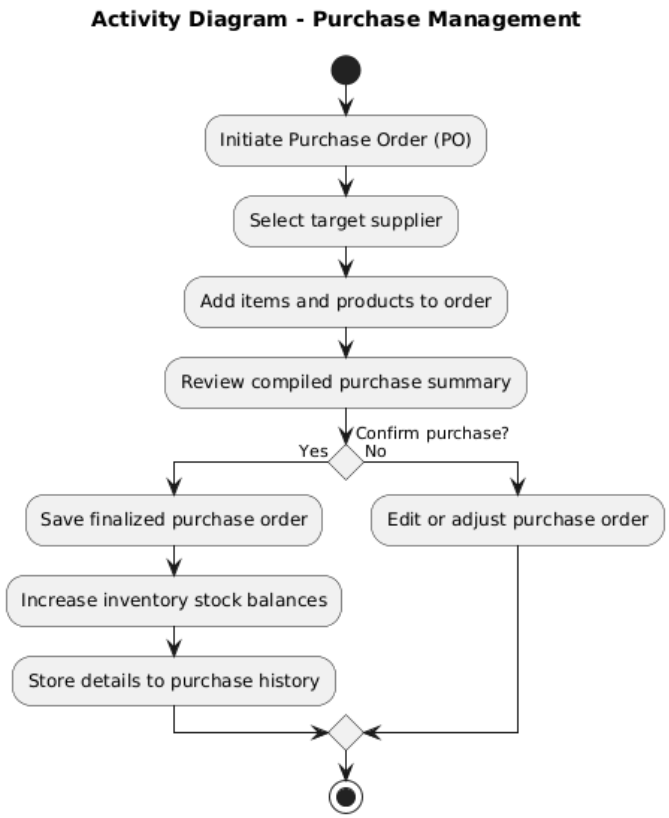

Figure 1.3.5: Activity Diagram of Purchase Management

1.3.6 User Report Generation

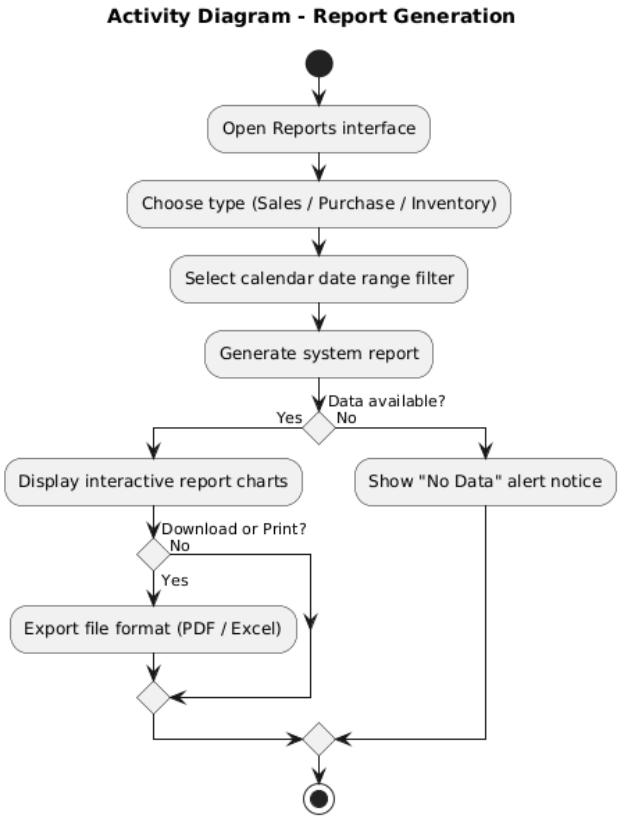

Figure 1.3.6: Activity Diagram of Report generation

1.4 Sequence Diagram

1.4.1 User Login

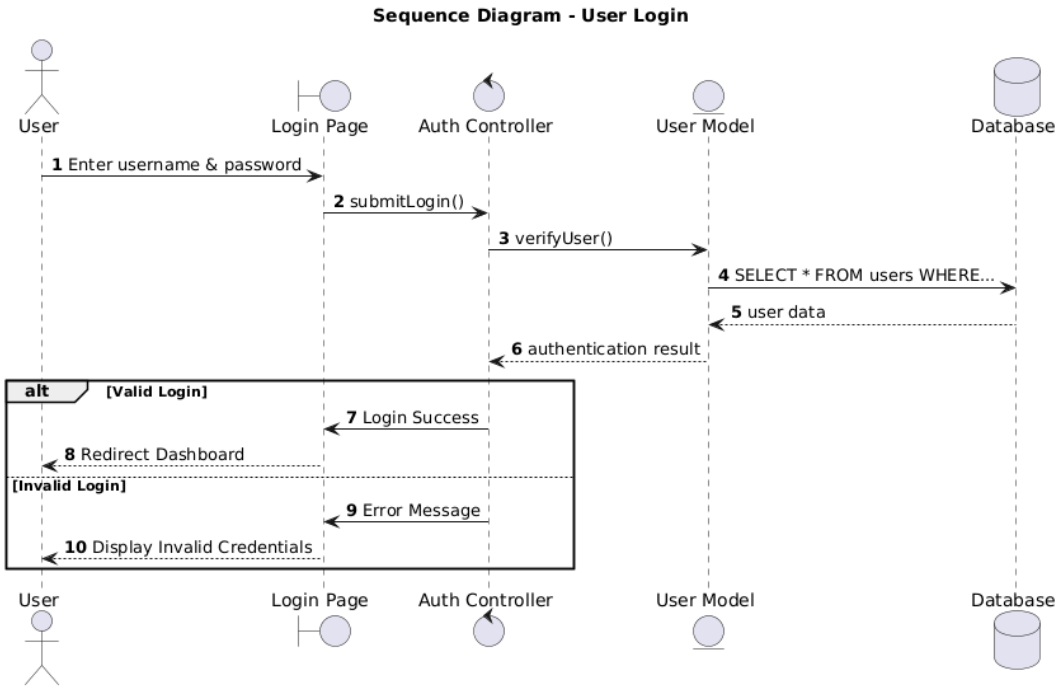

Figure 1.4.1: Sequence Diagram of User Login

1.4.2 Manage Users

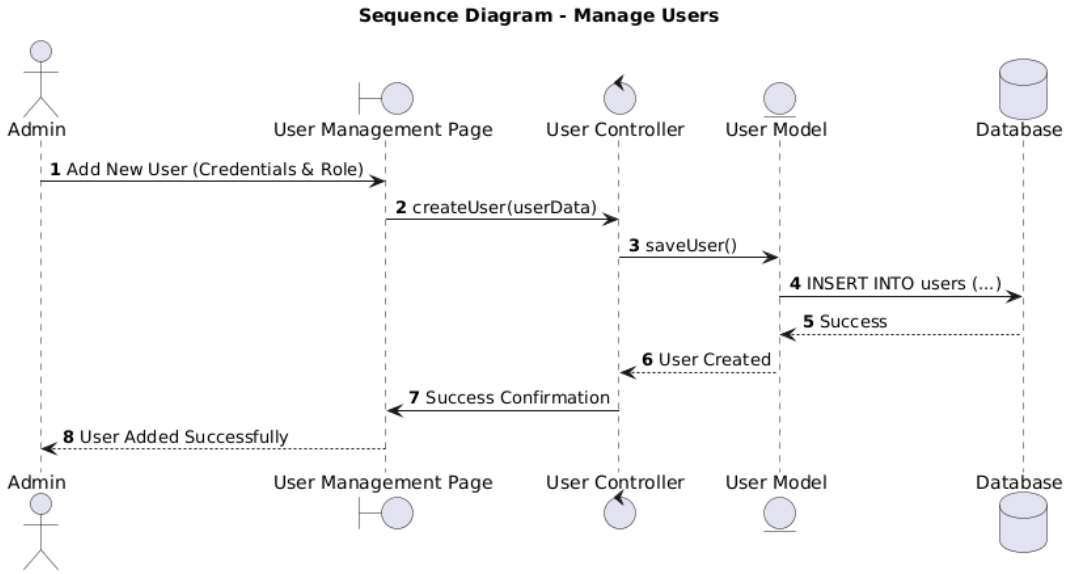

Figure 1.4.2: Activity Diagram of Manage Users

1.4.3 Product Management

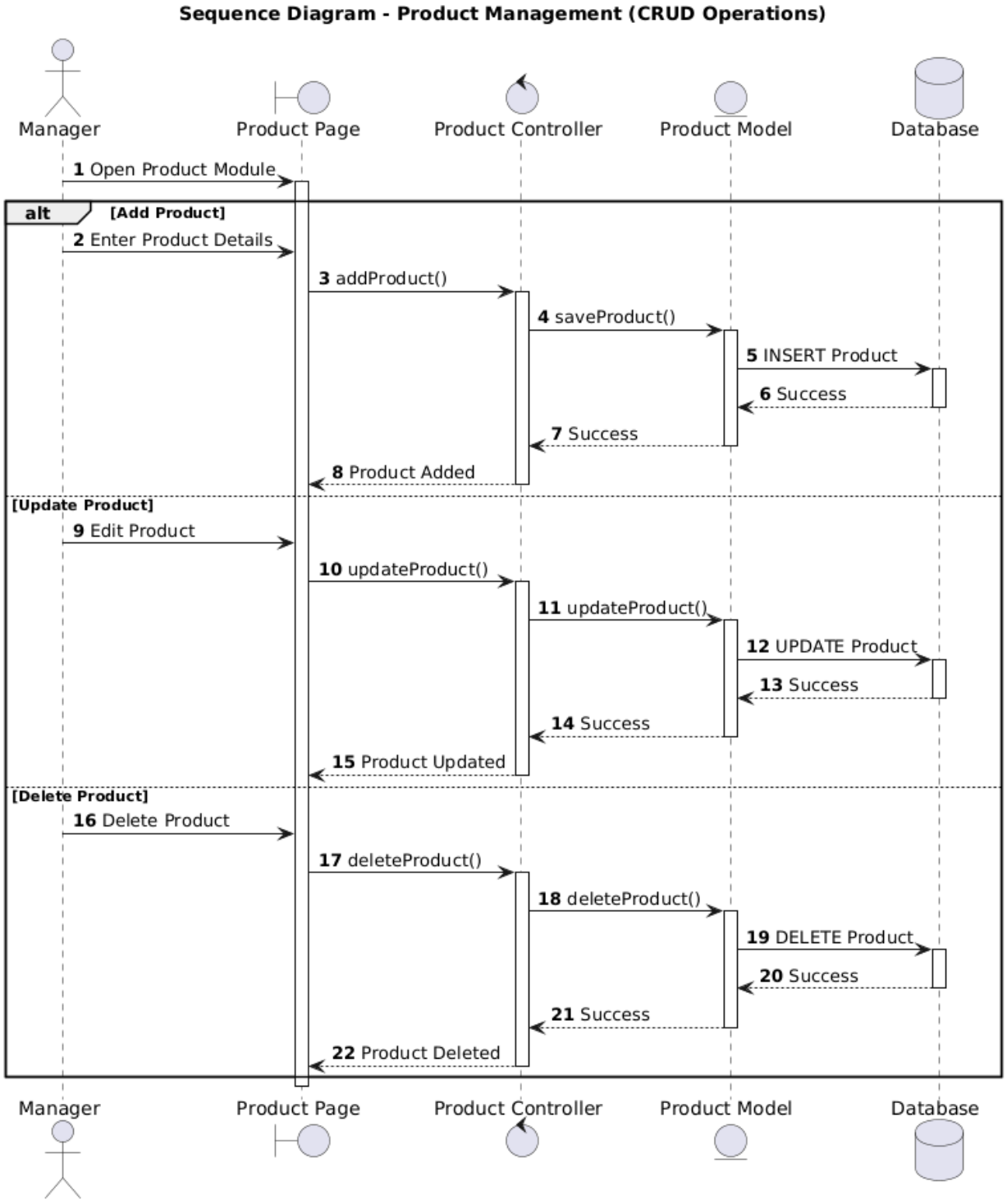

Figure 1.4.3: Activity Diagram of Product Management

1.4.4 Sell Products

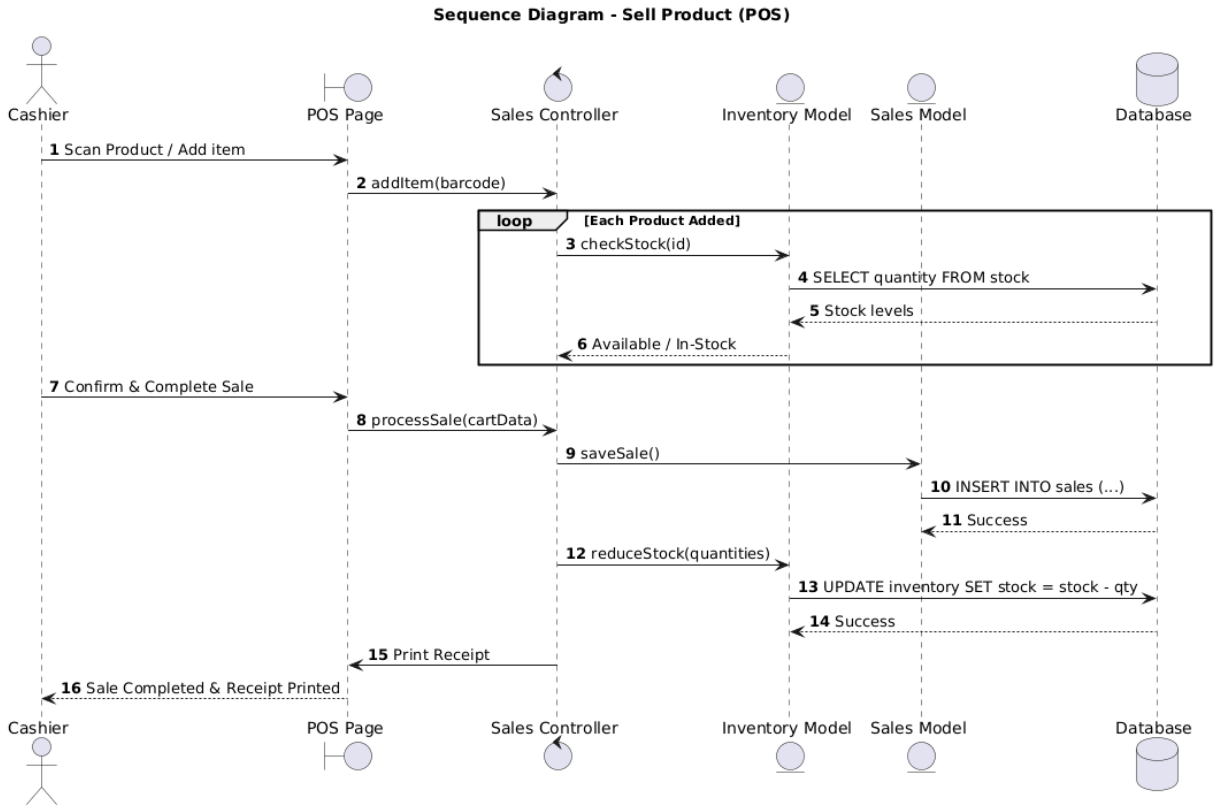

Figure 1.4.4: Activity Diagram of Sale Products

1.4.5 Purchase Stock

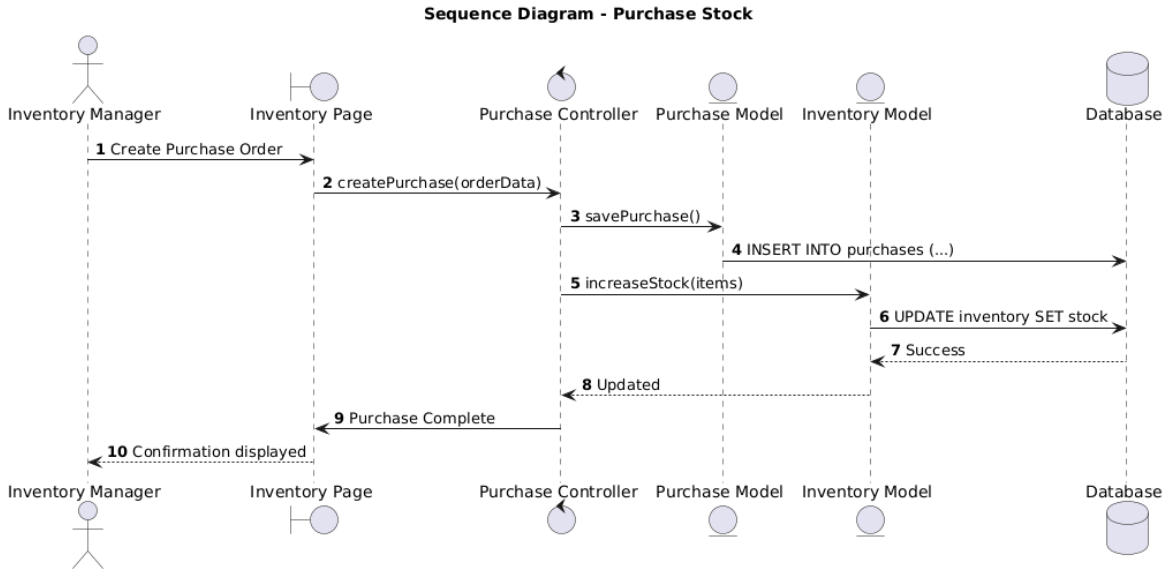

Figure 1.4.5: Activity Diagram of Purchase Stock

1.4.6 Report Generation

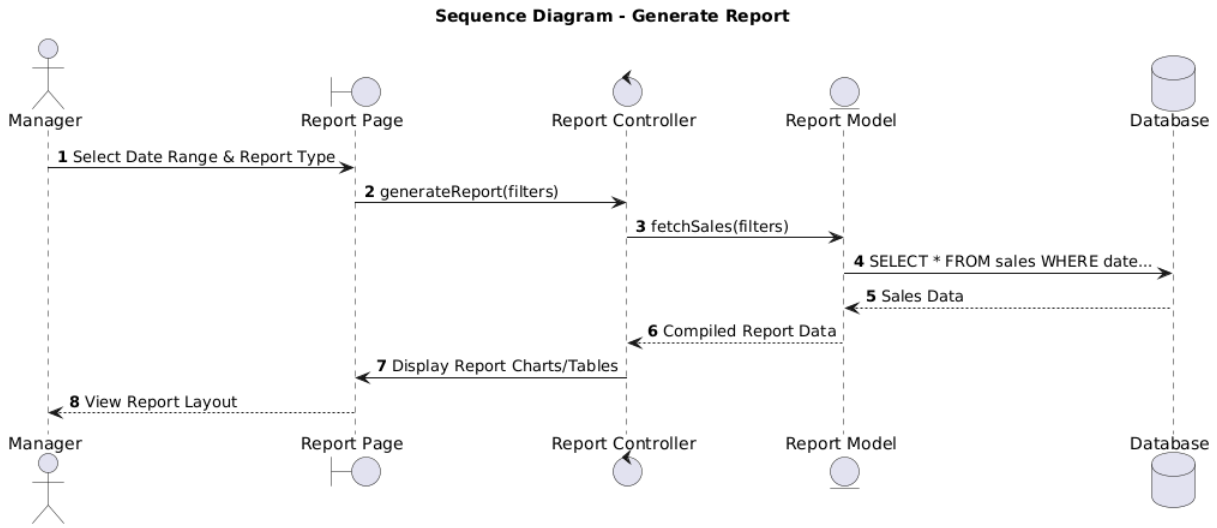

Figure 1.4.6: Activity Diagram of Report generation

1.4.7 Manage Supplier

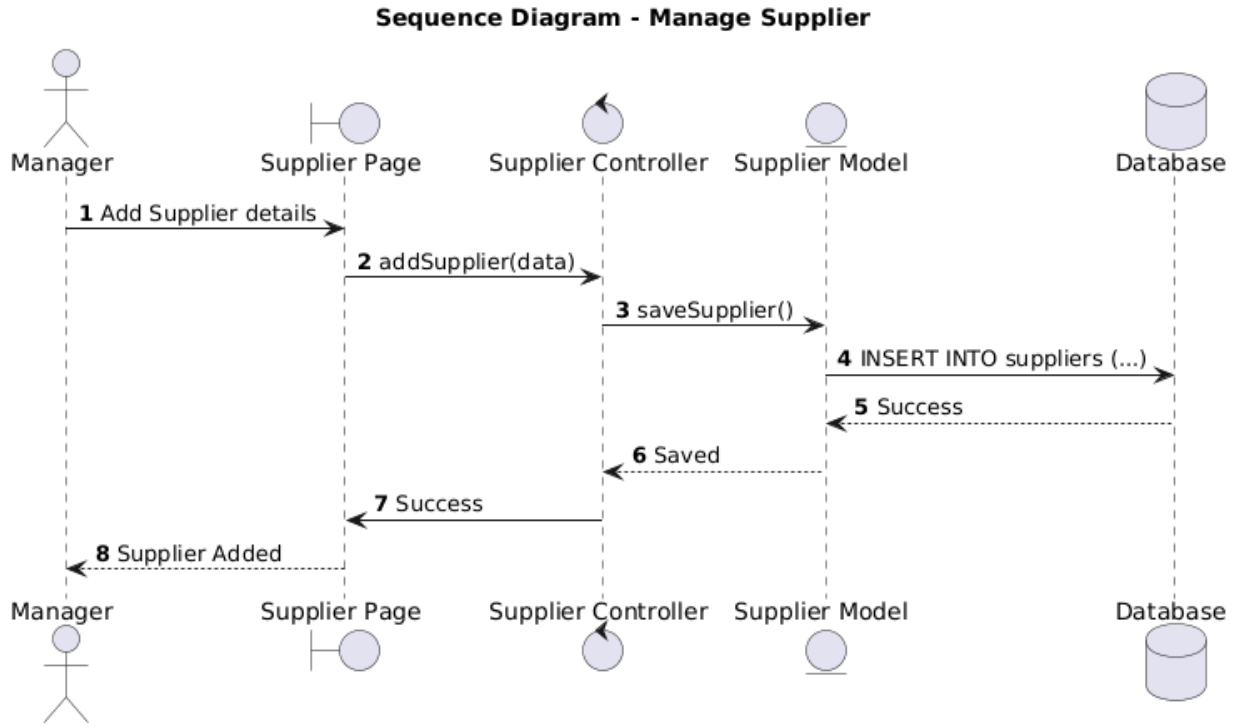

Figure 1.4.7: Activity Diagram of Manage Supplier

1.5 Flow Models

1.5.1 Level 0 DFD (Context Diagram) — Revised

The context diagram shows “Sales & Inventory Tracking System” as a single process interacting with four external entities: Customer, Cashier, Admin, and Supplier.

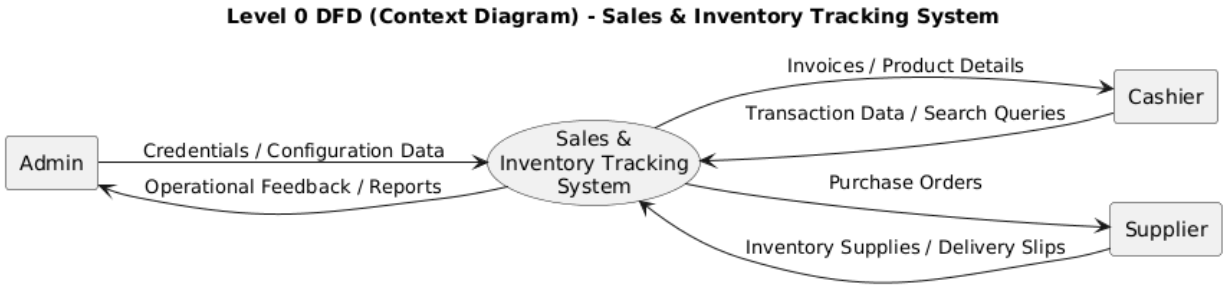

Figure 1.5.1: Level 0 DFD (Context Diagram) of Sales & Inventory Tracking System

1.5.2 Level 1 DFD — Revised

The Level 1 DFD decomposes the system into seven major processes — Authentication & User Management, Product Management, Inventory Management, POS/Sales Processing, Purchase Management, Supplier Management, and Reporting — along with their associated data stores.

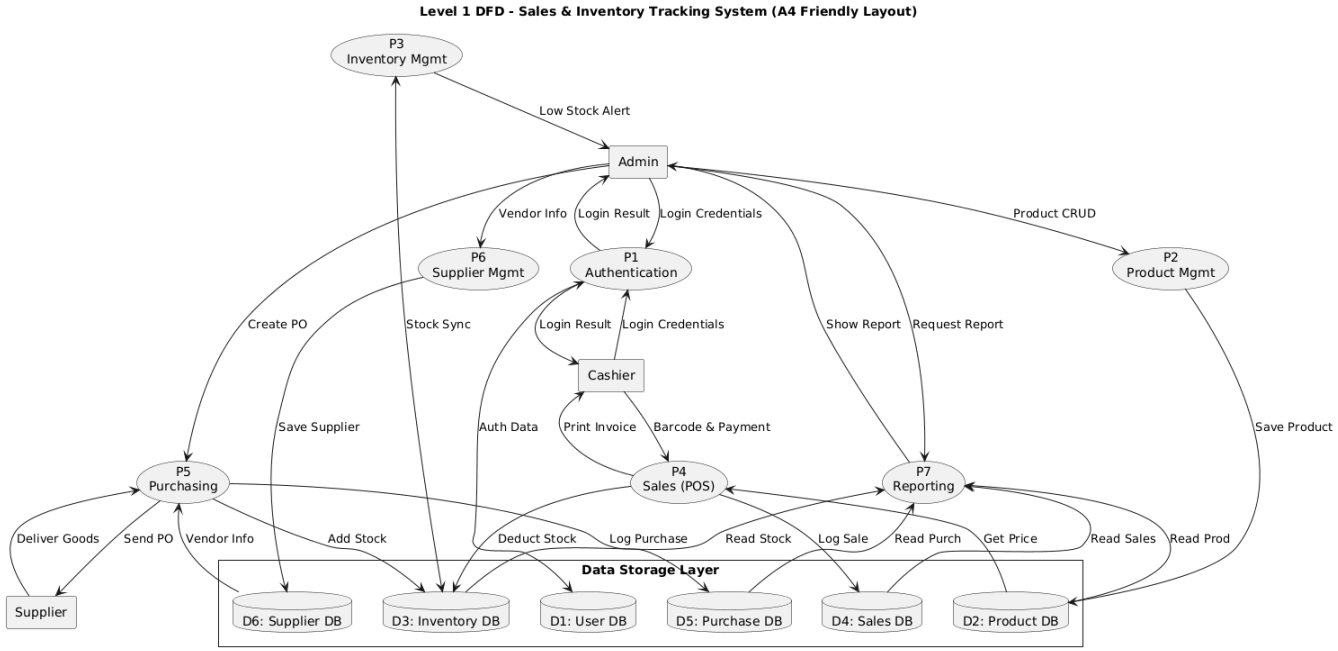

Figure 1.5.2: Level 1 DFD of Sales & Inventory Tracking System

1.5.3 Level 2 DFD

The Level 2 DFD decomposes the POS/Sales Processing process (Process 4.0) into its constituent sub-processes, from product search through invoice generation and inventory update.

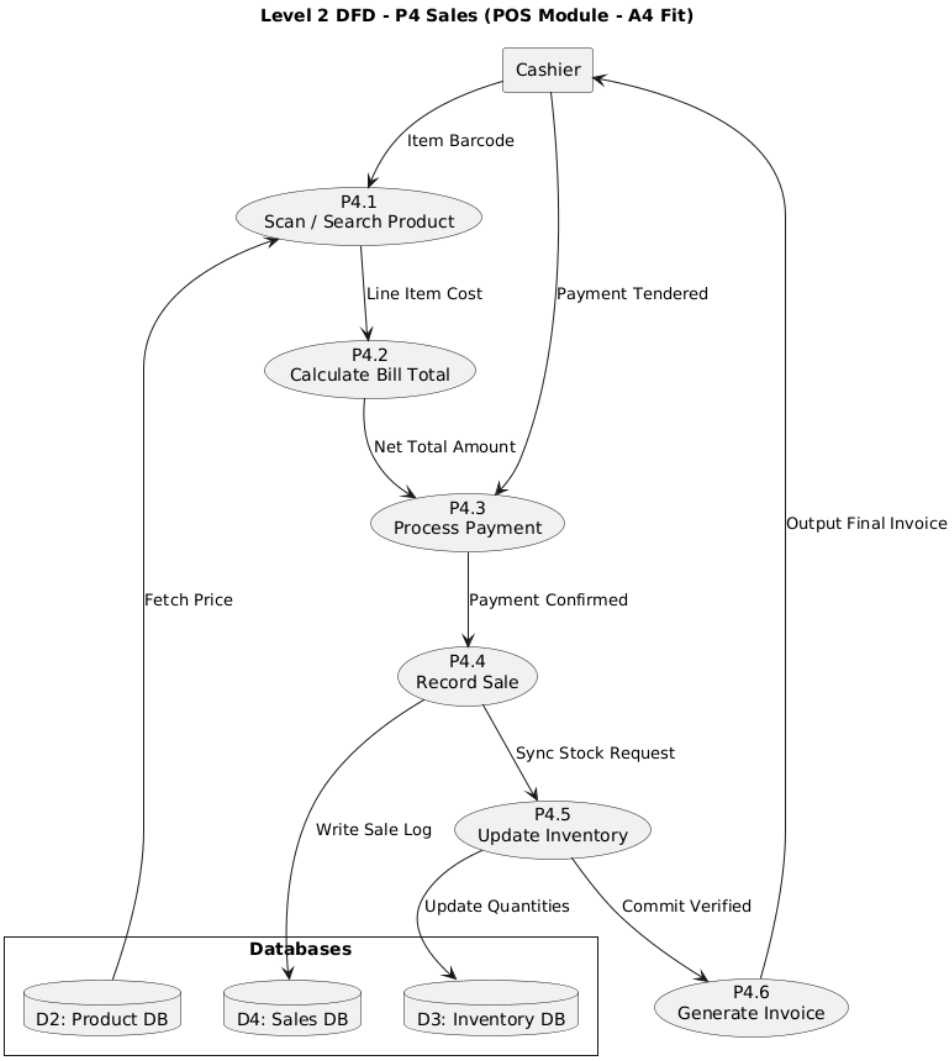

Figure 1.5.3: Level 2 DFD — Decomposition of POS/Sales Processing

1.6 Class-Based Models

1.6.1 ER Diagram

The Entity-Relationship diagram below models the core data entities of RBMS and their cardinalities.

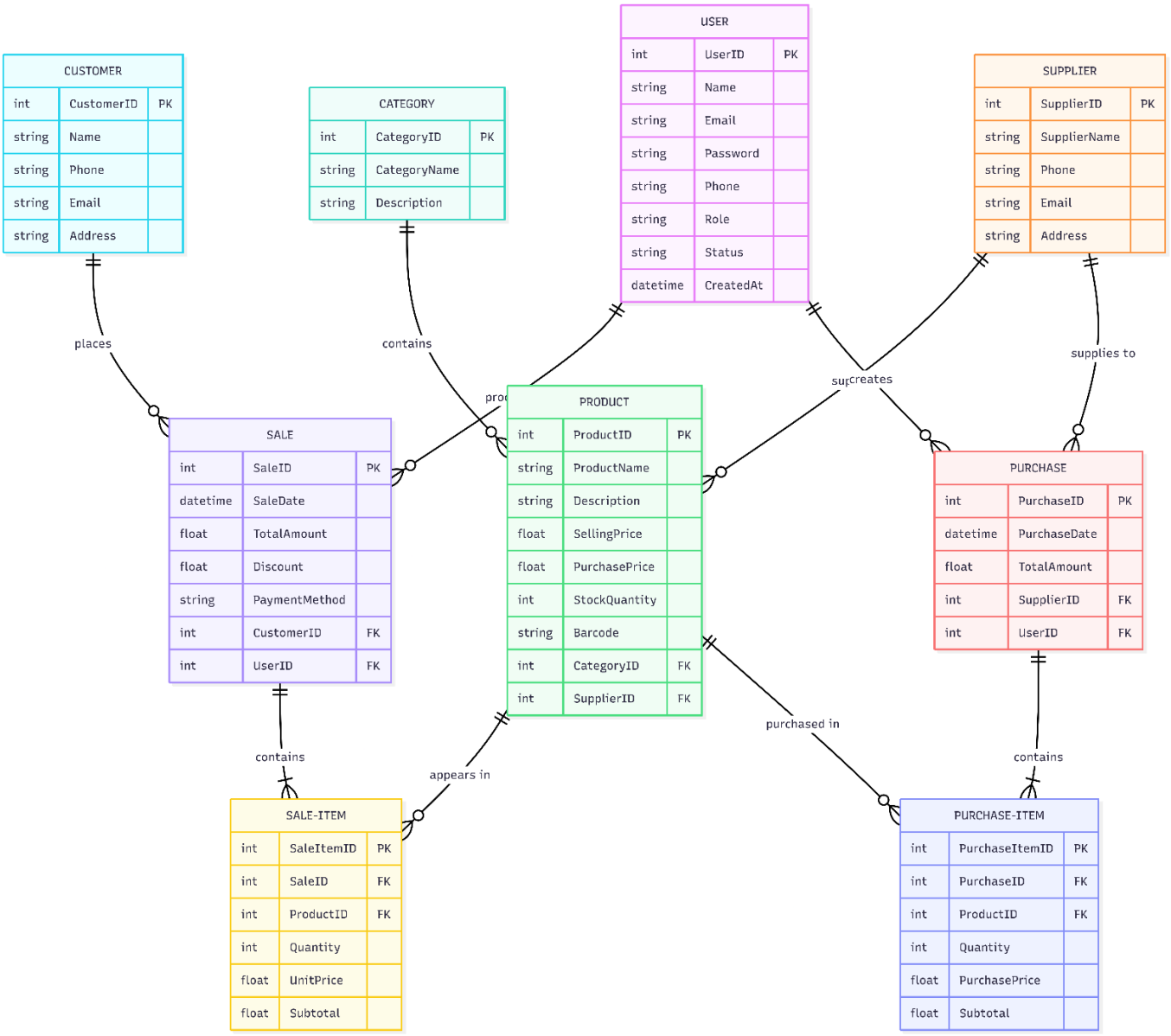

Figure 1.6.1: Entity-Relationship Diagram of Sales & Inventory Tracking System

1.6.2 Class Diagram

The UML class diagram below shows the primary classes of RBMS with their attributes, methods, and associations, consistent with the CRC cards in Section 1.2.3.

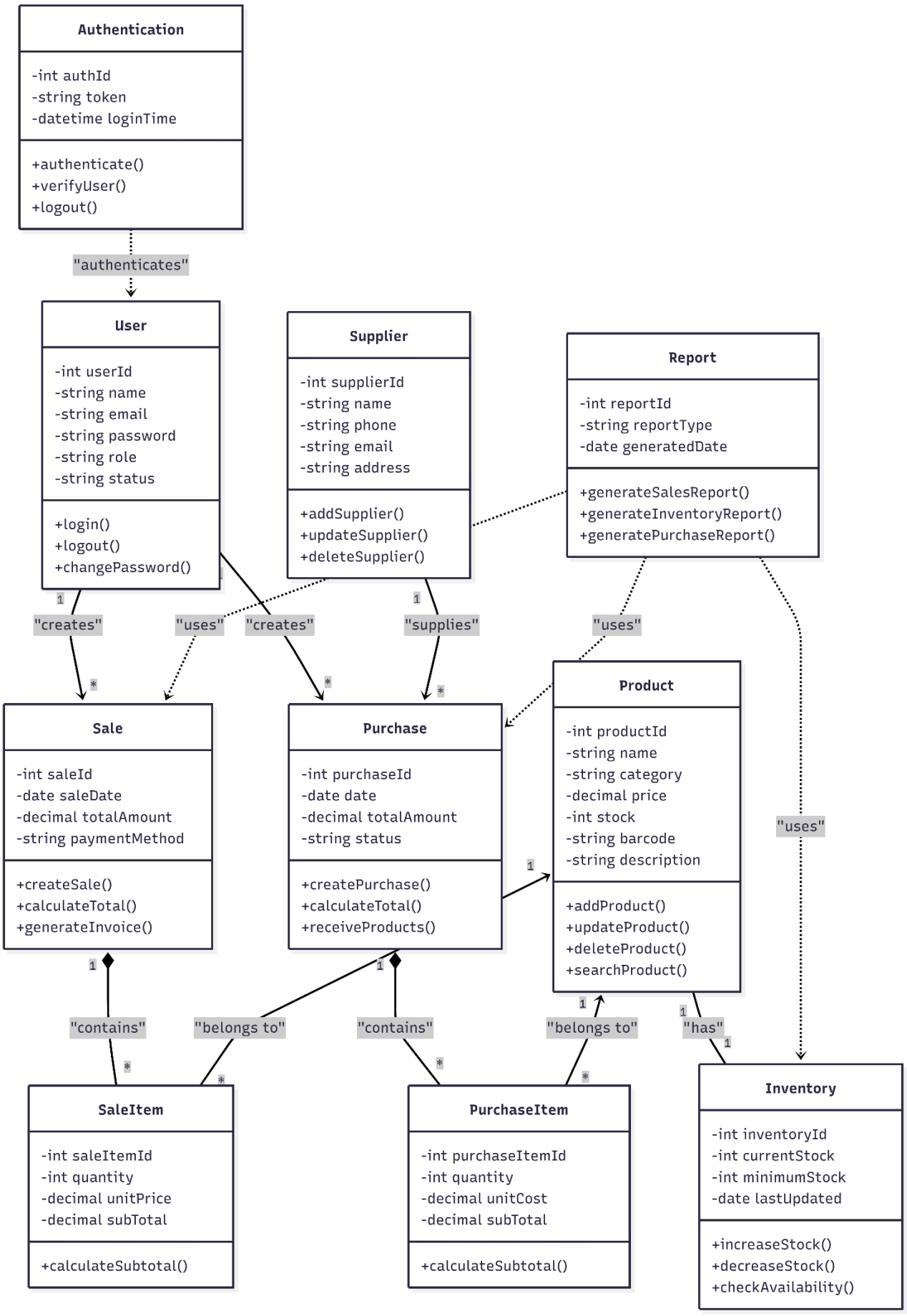

Figure 1.6.2: UML Class Diagram of Sales & Inventory Tracking System

1.6.3 CRC Modeling

Class-Responsibility-Collaborator (CRC) cards for the ten core classes of RBMS are given below.

Class	Responsibilities	Collaborators
User	Log in to the system; log out; manage personal profile; change password; perform operations according to assigned role.	Authentication, Role
Authentication	Verify user credentials; authenticate users; generate session/token; validate authenticated requests; handle logout.	User, Role
Role	Define user roles; manage permissions; authorize user actions.	User, Authentication
Product	Add, update, delete, and search products; maintain product pricing.	Category, Inventory, Purchase, Sale
Category	Create, update, and delete categories; organize products into categories (brand, gender, collection).	Product
Inventory	Maintain stock levels; increase stock after purchases; decrease stock after sales; monitor low-stock products.	Product, Purchase, Sale
Supplier	Store and update supplier information; maintain supplier records.	Purchase
Purchase	Create purchase orders; record purchased items; calculate purchase totals; update inventory after purchases.	Supplier, Product, Inventory
Sale (POS)	Process customer sales; calculate total amount; apply discounts; generate invoices; update inventory.	Product, Inventory, Report
Report	Generate sales, purchase, inventory, and profit reports.	Sale, Purchase, Inventory, Product

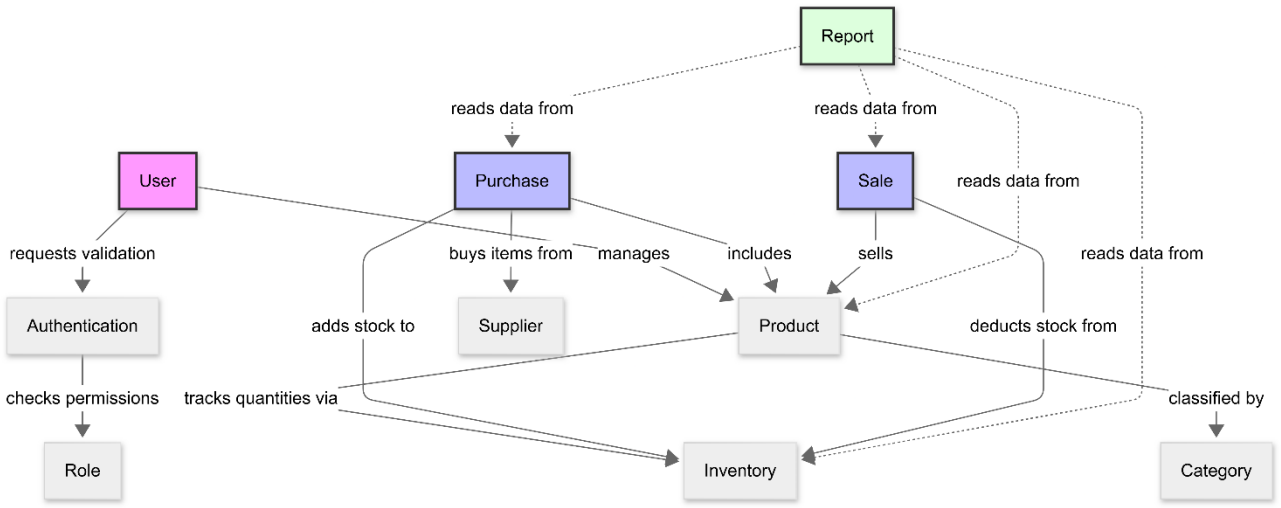

Figure 1.6.3: CRC Modeling of Sales & Inventory Tracking System

2. Project Planning

2.1 Work Breakdown Structure (WBS)

The project is decomposed into ten major activities (A–J), each further broken down into concrete tasks, as shown below.

- A. Requirement Gathering & Analysis
  - Stakeholder interviews
  - Functional / non-functional requirement listing
- B. SRS Documentation
  - Draft SRS
  - Review and finalize SRS
- C. System Design (DFD, ERD, UML, Architecture)
  - DFD (Level 0–2)
  - ER diagram
  - Class diagram & CRC cards
  - Architecture design
- D. Database Design
  - Schema design
  - Normalization
- E. Frontend Development
  - UI components
  - POS interface
  - Admin dashboard
- F. Backend Development
  - Authentication module
  - Product / Inventory module
  - Sales / Purchase module
  - Reporting module
- G. API Integration
  - Connect frontend to backend
  - Payment gateway integration (bKash / Nagad / card)
- H. Testing & Bug Fixing
  - Unit testing
  - Integration testing
  - Bug fixing
- I. Documentation & Report Writing
  - Compile report
  - Proofreading
- J. Final Presentation & Deployment
  - Prepare slides
  - Deploy to server

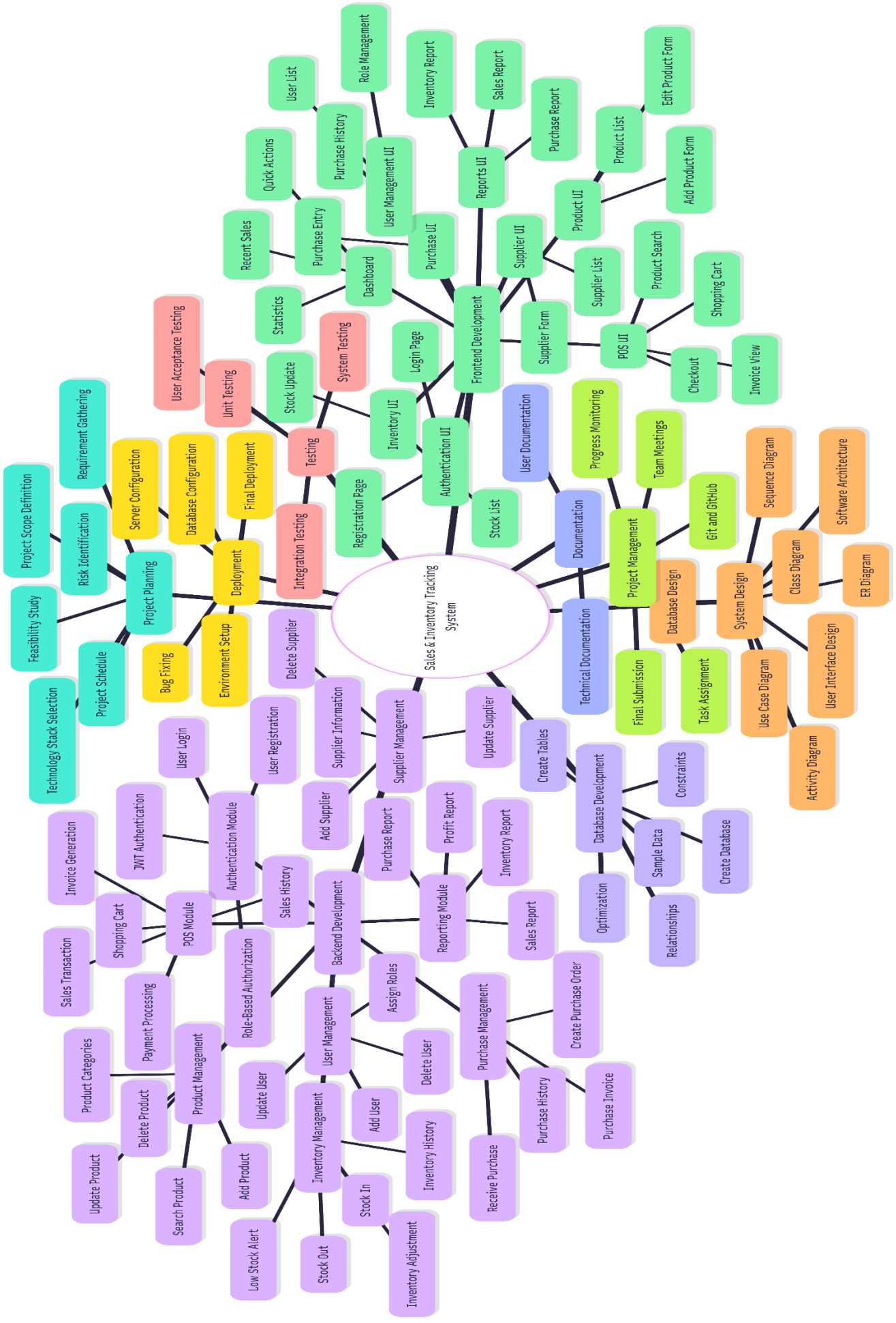
Sales & Inventory Tracking System
Project Planning
Project Scope Definition
Feasibility Study
Technology Stack Selection
Requirement Gathering
Risk Identification
Project Schedule
Deployment
Server Configuration
Database Configuration
Final Deployment
Bug Fixing
Environment Setup
Testing
Unit Testing
User Acceptance Testing
System Testing
Integration Testing
Frontend Development
Login Page
Registration Page
Dashboard
Recent Sales
Statistics
Quick Actions
Purchase Entry
Purchase History
User List
Role Management
User Management UI
Inventory UI
Stock Update
Stock List
Authentication UI
Purchase UI
Inventory Report
Sales Report
Purchase Report
Reports UI
Supplier UI
Supplier Form
Supplier List
Product UI
Product List
Edit Product Form
Add Product Form
POS UI
Product Search
Shopping Cart
Checkout
Invoice View
Backend Development
Authentication Module
User Login
JWT Authentication
User Registration
Role-Based Authorization
Update User
Add User
Delete User
Assign Roles
POS Module
Invoice Generation
Shopping Cart
Sales Transaction
Payment Processing
Sales History
Product Management
Product Categories
Delete Product
Update Product
Add Product
Search Product
Inventory Management
Low Stock Alert
Stock Out
Stock In
Inventory Adjustment
Inventory History
Supplier Management
Delete Supplier
Supplier Information
Add Supplier
Update Supplier
Purchase Management
Create Purchase Order
Purchase Invoice
Purchase History
Receive Purchase
Reporting Module
Profit Report
Purchase Report
Inventory Report
Sales Report
Database Development
Create Tables
Constraints
Optimization
Sample Data
Relationships
Create Database
Technical Documentation
Documentation
User Documentation
Project Management
Progress Monitoring
Team Meetings
Git and GitHub
Task Assignment
Final Submission
Database Design
System Design
Sequence Diagram
Class Diagram
ER Diagram
Use Case Diagram
Activity Diagram
Software Architecture
User Interface Design

Figure 2.1: WBS

2.2 Project Scheduling: CPM and Gantt Chart

Activity durations and dependencies were used to compute the Critical Path using forward and backward pass calculations.

2.2.1 Activity Table

Activity	Predecessor	Duration (Weeks)
A	-	2
B	A	1
C	B	2
D	C	1
E	C	3
F	D	4
G	E, F	2
H	G	2
I	H	1
J	I	1

2.2.2 Forward and Backward Pass

Activity	ES	EF	LS	LF	Slack
A	0	2	0	2	0
B	2	3	2	3	0
C	3	5	3	5	0
D	5	6	5	6	0
E	5	8	7	10	2
F	6	10	6	10	0
G	10	12	10	12	0
H	12	14	12	14	0
I	14	15	14	15	0
J	15	16	15	16	0

2.2.3 CPM Network Diagram

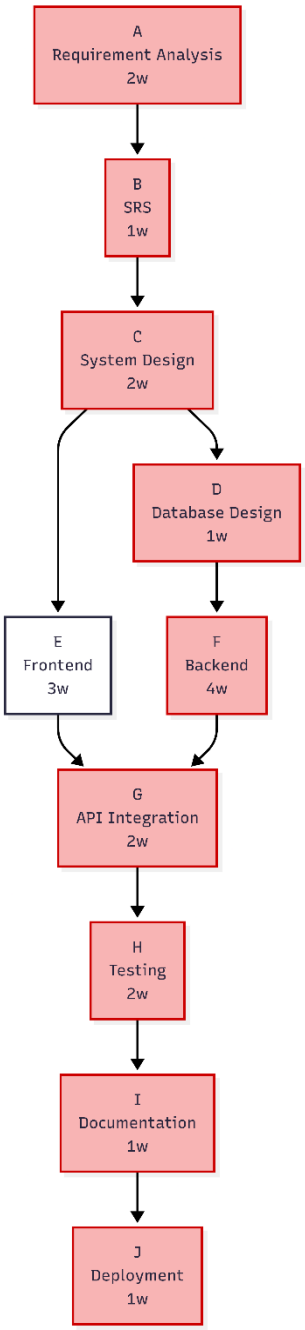

Figure 2.2: CPM Network Diagram (AON) — Critical path shown in red

Critical Path: A → B → C → D → F → G → H → I → J

Total Project Duration = 16 Weeks. Activity E (Frontend Development) has 2 weeks of slack and can be delayed without affecting the overall completion date.

2.2.4 Gantt Chart

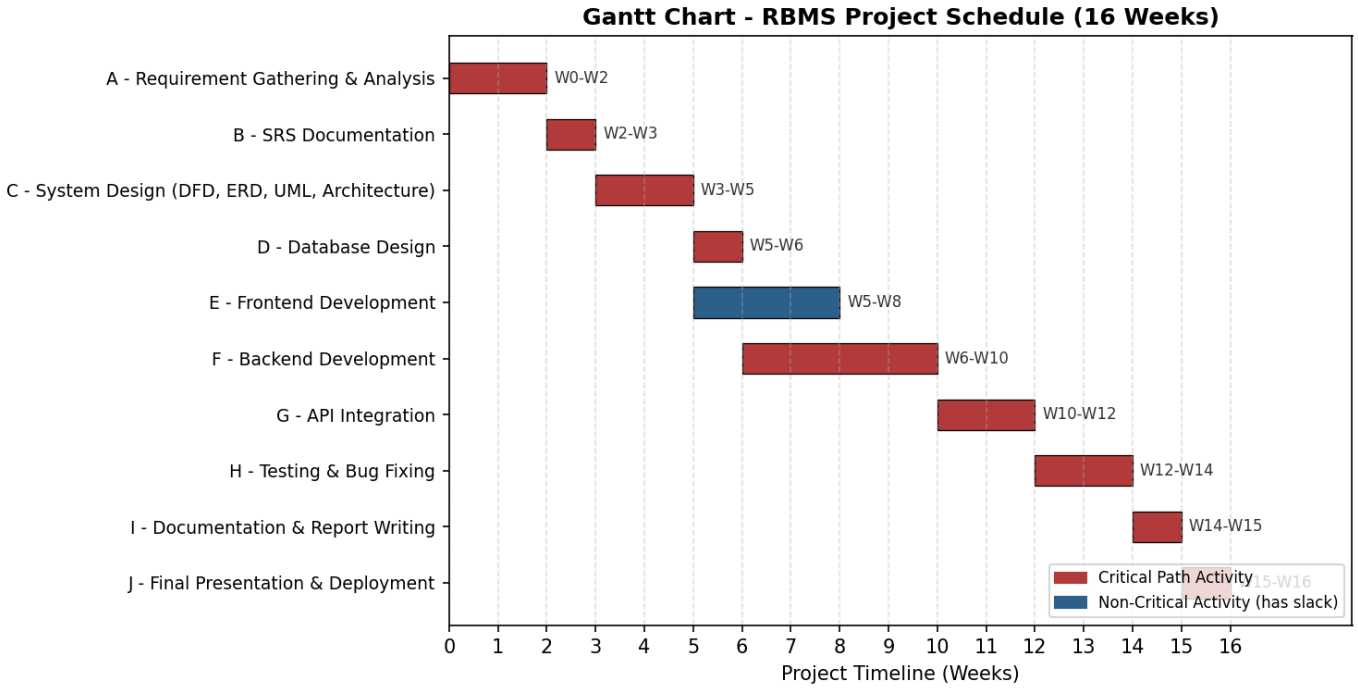

Figure 2.3: Gantt Chart of the RBMS Project Schedule

2.3 Project Estimation

Project size, effort, schedule, and cost were estimated using Function Point Analysis (FPA) combined with the Basic COCOMO model, which is appropriate for a small, well-understood application developed by a small team (the "organic" mode).

2.3.1 Product Size Estimation (Function Point Analysis)

Component	Count	Avg. Weight	Function Points
External Inputs (EI)	10	4	40
External Outputs (EO)	9	5	45
External Inquiries (EQ)	4	4	16
Internal Logical Files (ILF)	11	10	110
External Interface Files (EIF)	1	7	7

- Unadjusted Function Points (UFP) = 40 + 45 + 16 + 110 + 7 = 218 FP
- Value Adjustment Factor (VAF) ≈ 1.05 (moderate complexity: multi-role access, real-time inventory sync, payment integration)
- Adjusted Function Points = 218 × 1.05 ≈ 229 FP
- Using an average of 55 LOC/FP for a JavaScript/Node.js stack: Estimated Size ≈ 229 × 55 ≈ 12,595 LOC ≈ 12.6 KLOC

Justification: The count of Internal Logical Files reflects the eleven core data entities identified in the ER diagram (User, Role, Product, Category, Inventory, Supplier, Purchase, Purchase Item, Sale, Sale Item, Invoice/Report). External Outputs are dominated by the seven report types required in Section 4.1.7 of the SRS plus invoice and low-stock alert. The single External Interface File represents integration with third-party mobile banking gateways (bKash, Nagad).

2.3.2 Effort Estimation (Basic COCOMO — Organic Mode)

Effort formula: E = a × (KLOC)^b person-months, where a = 2.4 and b = 1.05 for organic-mode projects (small team, familiar environment, flexible requirements).

- E = 2.4 × (12.6)^1.05 ≈ 2.4 × 14.3 ≈ 34.3 person-months

Justification: RBMS fits the organic category because the team is small (2 developers), the application domain (retail POS) is well understood, and requirements were fixed early through the SRS.

2.3.3 Schedule Estimation

Schedule formula: D = c × (E)^d months, where c = 2.5 and d = 0.38 for organic-mode projects.

- D = 2.5 × (34.3)^0.38 ≈ 2.5 × 3.83 ≈ 9.6 months (≈ 42 weeks) — theoretical full-scale commercial estimate

Justification / Reconciliation with the Actual Plan: The COCOMO estimate of ~9.6 months applies to a fully-featured, production-grade commercial system built to industry robustness standards. The academic project plan (Section 2.2) targets a 16-week (4-month) timeline because: (a) the scope delivered for the course is a functional prototype covering the core POS, inventory, purchase, and reporting workflows rather than every enterprise feature (accounting, HR, CRM are deferred to "Future Enhancements"); (b) some non-functional hardening (extensive load testing, full multi-branch scalability) is out of scope for the lab deliverable; and (c) the two-person team works in parallel and reuses well-known frameworks (React/Next.js, Node/Express, PostgreSQL), which reduces ramp-up time relative to the COCOMO organic-mode default assumptions.

2.3.4 Cost Estimation

- Theoretical full-scale cost = Effort × average developer rate = 34.3 person-months × BDT 40,000/month ≈ BDT 13,72,000 (≈ 13.7 lakh Taka), if built and staffed as a commercial product.
- Academic project cost (actual): limited to development-support costs only — hosting/domain (≈ BDT 2,000), cloud database tier (≈ BDT 1,500), and payment-gateway sandbox (free tier) — since student effort is unpaid course work.

Justification: The cost gap mirrors the schedule gap in Section 2.3.3 — the monetary figure from COCOMO represents what it would cost to hire a team to build a production RBMS, which is useful as a size/complexity benchmark, while the academic project's real cash outlay is limited to infrastructure needed to demo the prototype.

3. Risk Analysis

3.1 SWOT Analysis

Strengths

- Centralized, real-time system replacing manual/disconnected record keeping
- Modular architecture allows future expansion (Accounting, HR, CRM)
- Automated inventory and sales calculation reduces human error
- Role-based access control improves data security

Weaknesses

- Small development team (2 members) limits parallel development speed
- Fixed 16-week academic timeline constrains depth of testing
- Limited prior experience with large-scale system development
- Dependency on third-party mobile banking APIs (bKash, Nagad) for payments

Opportunities

- Growing demand for digitized retail/POS management among small businesses in Bangladesh
- Extendable to e-commerce, mobile app, and AI-based sales forecasting
- Potential for multi-branch retail chain adoption
- Cloud deployment enables a future SaaS subscription model

Threats

- Data security and privacy risks around payment and customer data
- Competition from established, feature-rich commercial POS software
- Regulatory changes affecting digital payment integrations
- Rapid technology change requiring continuous framework updates

3.2 RMMM Plan (Risk Mitigation, Monitoring, and Management)

The following original risks were identified for the RBMS project, with mitigation, monitoring, and management strategies for each.

Risk	Mitigation	Monitoring	Management
Requirement volatility / scope creep	Freeze SRS after stakeholder sign-off; require formal change requests for new features.	Weekly review of progress against the SRS baseline.	Evaluate change requests for impact; renegotiate scope or timeline if accepted.
Schedule slip on critical-path activities (e.g., Backend Development)	Build in slack (Activity E has 2 weeks); parallelize non-dependent tasks.	Track actual vs. planned progress on the CPM critical path every week.	Reallocate developer time or extend working hours to recover the critical path.
Payment gateway (bKash/Nagad) integration failure	Start integration early using the sandbox/test environment; study API documentation in advance.	Set integration milestones and test transactions weekly during Phase G.	Fall back to manual payment recording in the POS if integration is not complete by deadline.
Team/resource risk (illness or unavailability of a 2-person team)	Cross-train both members on all modules; maintain up-to-date shared documentation.	Short daily/weekly stand-up check-ins.	Redistribute tasks between the two members; request a short deadline extension if necessary.
Data security risk (unauthorized access, credential leakage)	Use JWT-based authentication, bcrypt password hashing, and strict role-based authorization.	Conduct basic security testing (auth bypass, SQL injection checks) before deployment.	Patch vulnerabilities immediately and rotate any exposed credentials.
Database performance risk under concurrent POS transactions	Normalize schema, add indexes on frequently-queried fields (barcode, product_id).	Monitor query response time during integration testing with sample large datasets.	Optimize slow queries and introduce caching for read-heavy report queries.
Team unfamiliarity with chosen frontend/backend framework	Allocate a short learning period at project start; use official documentation and tutorials.	Track task completion velocity in the first two weeks of development.	Switch to a simpler, more familiar stack if progress stalls significantly.
System not matching real retail workflow expectations	Validate requirements against real store workflows; involve a mock stakeholder for feedback.	Hold periodic demo/review sessions at the end of each major phase.	Incorporate feedback into the next iteration before final submission.